



School of Future Tech

Case Study Report

on

SystemFitnessMembershipSystem

by

Name:Arnesh Munj

Roll no:150096725034

About me: B.Tech 2025-2029 Backend Project

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.

6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Campus media teams publish news, event coverage and opinion pieces, and these usually need an editor's approval before they reach readers. Without a controlled workflow, unreviewed drafts can become public, and any contributor could change another contributor's work.

This case study builds a full-stack News and Article Publishing Platform that handles this workflow. Authors write articles as drafts, editors review and publish them, and readers can search and filter only the published articles. The backend is a REST API built with Node.js, Express.js and MongoDB (Atlas). The frontend is a React application that consumes the API. The project focuses on backend concerns: schema design, authentication, role-based and ownership-based authorization, and a status-driven workflow.

2. Problem Statement / Case Background (Abstract)

A campus media team wants authors to draft articles that must be reviewed and published by an editor before they become publicly visible. Authors may edit only their own drafts, and readers must be able to search published articles by category.

The objectives are to:

- Design Author, Editor and Article schemas using Mongoose.
- Implement a draft, review and publish workflow using a `status` field.
- Implement ownership-based authorization for draft editing.
- Implement role-based authorization so that only editors can publish.
- Implement search and category-based filtering for published articles.

The expected outcomes are that authors can create and edit only their own drafts, only editors can approve and publish an article, readers can search and filter published articles by category, and unpublished drafts remain hidden from the public listing.

3. Case Study Design

3.1 System architecture

The system has three layers: a React single-page application (client), an Express.js REST API (server), and a MongoDB Atlas database. The client sends HTTP requests with a JSON Web Token (JWT) in the **Authorization** header. The server verifies the token, applies authorization middleware, runs the controller logic, and queries the database through Mongoose.

React (Vite) → Axios → Express API → Middleware → Controller → Mongoose → MongoDB Atlas

3.2 Roles

Role	Permissions
Reader (no login)	View, search and filter published articles
Author	Create articles (saved as drafts), edit own drafts, view own articles
Editor	View all drafts, publish a draft

3.3 Database design

The Author and Editor are implemented as a single **User** collection that has a **role** field. This avoids duplicate schemas and login logic.

User: **name**, **email** (unique, lowercase), **password** (bcrypt hash), **role** (author or editor, default author), **createdAt**, **updatedAt**.

Article: **title**, **content**, **category**, **status** (draft or published, default draft), **author** (reference to User), **publishedAt**, **createdAt**, **updatedAt**.

The **author** reference in the **Article** is what makes ownership checks possible.

3.4 Workflow

An article is created with the status **draft**. Its owner can edit it any number of times while it stays in that state. An editor then publishes it, which sets the status to **published** and records **publishedAt**. After that point the article is public and can no longer be edited.

3.5 API design

Meth od	Endpoint	Access	Purpose
POST	/auth/register	Public	Create an account
POST	/auth/login	Public	Log in and receive a JWT

POST	/articles	Author	Create a draft
PATCH	/articles/:id	Owner author, draft only	Edit a draft
PATCH	/articles/:id/publish	Editor	Publish a draft
GET	/articles?category=&search=	Public	List published articles with filters
GET	/articles/:id	Public	View one published article
GET	/articles/my	Author	List the author's own articles
GET	/articles/drafts	Editor	List drafts awaiting review

The last three rows go beyond the four article endpoints required by the problem statement. They were added so that the frontend's details page, author dashboard and editor panel have the data they need.

3.6 Request flow

For a protected route, the request passes through the JWT authentication middleware, then the role check, then (for edits) the ownership check, and finally the controller.

Request → auth (verify JWT) → authorizeRoles → ownership → controller → response

3.7 Folder structure

```

NewsAndArticlePlatform/
├── backend/
│   ├── config/db.js
│   ├── models/user.js, Article.js
│   ├── middleware/auth.js, role.js, ownership.js
│   ├── controllers/authController.js, articleController.js
│   ├── routes/authRoutes.js, articleRoutes.js
│   └── index.js
└── frontend/NewAndArticle/
    └── src/api, components, context, pages

```

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

4.1 Technology stack

Layer	Technology	Purpose
Runtime	Node.js	JavaScript runtime for the server
Framework	Express.js 5	Routing and middleware
Database	MongoDB Atlas	Cloud-hosted document database
ODM	Mongoose 9	Schemas, validation and queries
Security	bcryptjs, jsonwebtoken	Password hashing and token authentication
Config	dotenv, cors	Environment variables and cross-origin access
Frontend	React with Vite	User interface
Frontend libraries	react-router-dom, Axios	Routing and API calls
Testing	Thunder Client	API testing
Deployment	Render (backend), Vercel (frontend)	Hosting

4.2 Methods and algorithms

Password hashing. Passwords are hashed with bcrypt (10 salt rounds) before being stored, so plain-text passwords never reach the database. At login, bcrypt.compare checks the entered password against the stored hash.

JWT authentication. On a successful login the server signs a JWT containing the user's id and role, with a one-day expiry. The authentication middleware reads the Authorization: Bearer <token> header, verifies the signature, and attaches req.user to the request. The server is stateless, so no sessions are stored.

Role-based authorization. A reusable middleware, authorizeRoles(...roles), compares req.user.role against the allowed roles and returns 403 Forbidden on a mismatch. It guards the publish, draft-listing and article-creation routes.

Ownership-based authorization. The ownership middleware loads the article and checks two conditions. First, the article's author must equal the logged-in user's id. Second, its status must still be draft. If either check fails, it returns 403.

Status-driven workflow. The status field acts as a small state machine (draft to published). The public queries always include status: "published" in their filter, so drafts are excluded at the database level and not only hidden in the UI.

Search and filtering. The listing endpoint builds a MongoDB filter from optional query parameters. `category` is matched case-insensitively and exactly. `search` is a case-insensitive pattern match on both title and content, combined with an `$or` condition. User input is escaped before it is used in the pattern, so special characters are treated as plain text and cannot cause regular expression errors or injection.

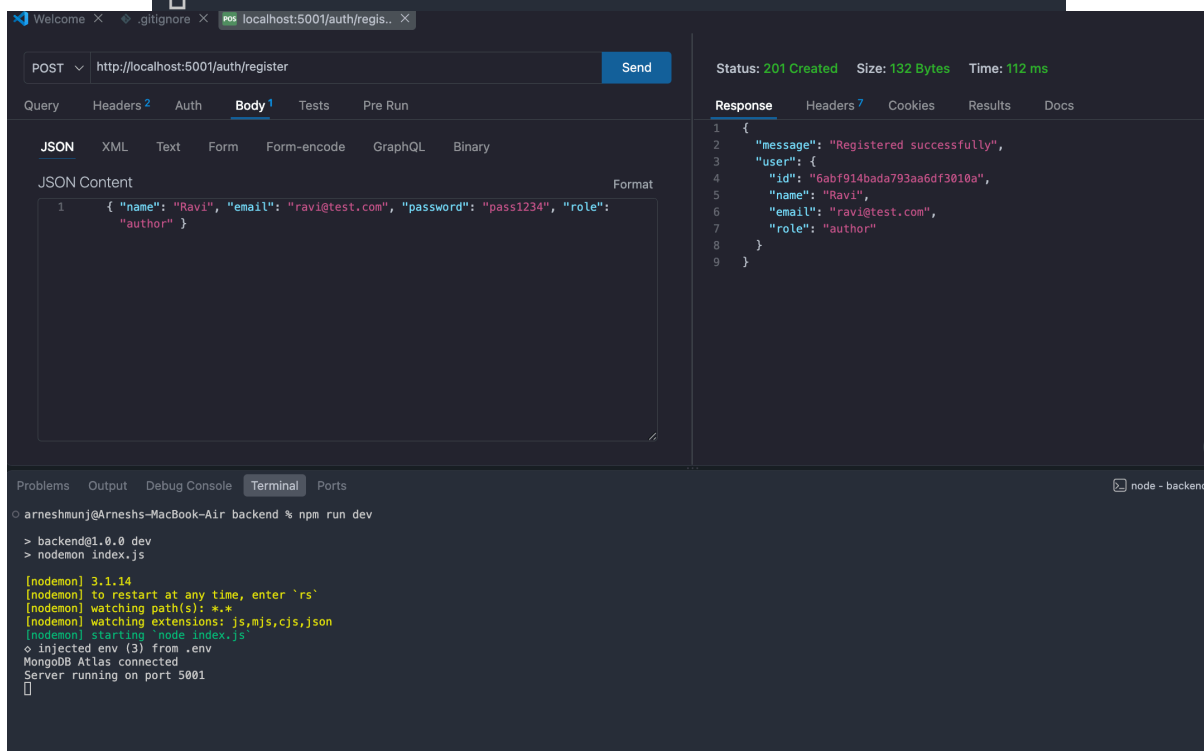
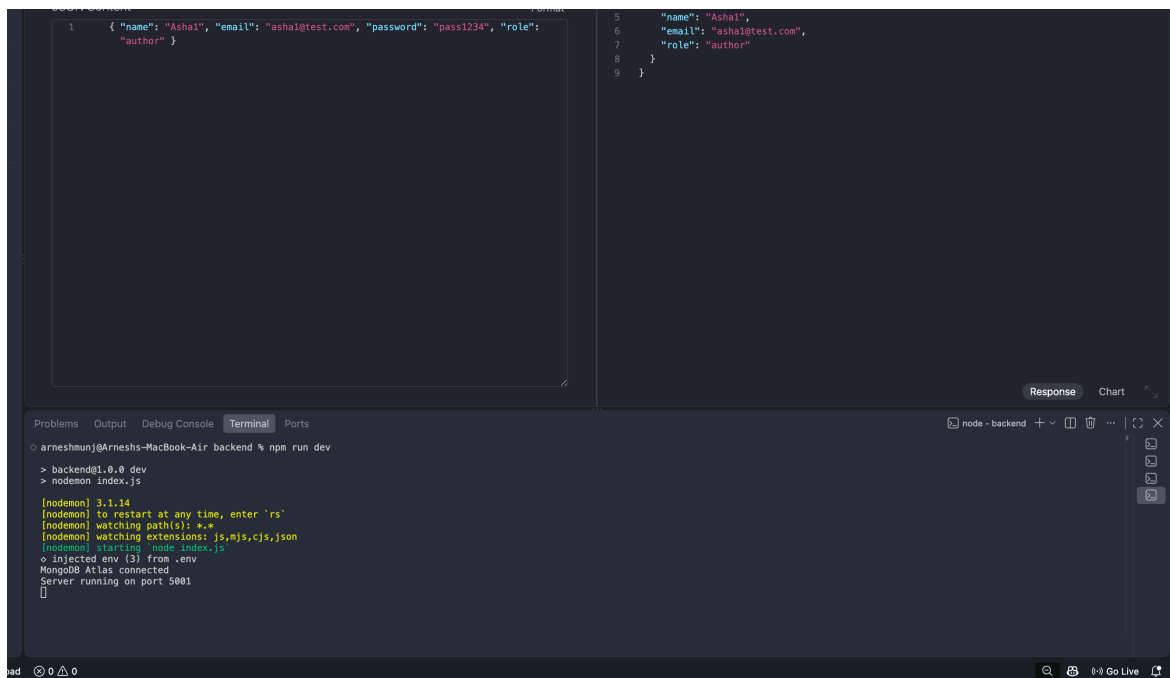
Validation and error handling. Controllers validate required fields, reject invalid `ObjectIds`, and return consistent JSON errors with suitable status codes (400, 401, 403, 404, 409).

Frontend authorization. A React Context stores the token and user, an Axios interceptor attaches the token to every request, and a `ProtectedRoute` component hides pages from users who lack the right role. The frontend checks only improve the user experience. All security decisions are enforced in the backend.

5. Problem Statement / Case Study Implementation Details

5.1 Implementation steps

1. Created a MongoDB Atlas cluster, a database user and a network access rule, and stored the connection string in `.env`.
2. Set up the Express server and a Mongoose connection module (`config/db.js`).
3. Defined the `User` and `Article` schemas.
4. Implemented registration and login with password hashing and JWT signing.
5. Wrote the authentication, role and ownership middleware.
6. Implemented the article controller and routes with search and category filtering.
7. Tested all endpoints in Thunder Client, including failure cases.
8. Built the React frontend: login and register, article list with search and filter, article form, details page, author dashboard and editor panel.
9. Deployed the backend to Render and the frontend to Vercel, with environment variables set in each platform's dashboard.



[illegible]

The screenshot shows the VS Code interface with a REST client extension. The top bar displays the method **GET** and the URL `http://localhost:5001/articles?category=Campus&search=festahost:5001/auth/login`. A **Send** button is visible. Below the URL bar, tabs for **Query**, **Headers**, **Auth**, **Body**, **Tests**, and **Pre Run** are shown. The **Body** tab is active, displaying **JSON** content. The JSON content area shows a single line: `1`. To the right of the JSON content area is a **Format** button. On the right side of the editor, the **Response** tab is active, showing the status **200 OK**, size **2 Bytes**, and time **24 ms**. The response body is `1 []`. At the bottom of the VS Code window, the **Terminal** tab is active, showing the command `node - backend`.

6. Problem Statement / Case Study Results and Conclusion.

6.1 Results

The following behaviours were verified through Thunder Client and the frontend:

Test	Expected result	Status
Register a new user	201 Created, password not returned	Pass
Register with an existing email	409 Conflict	Pass
Login with a wrong password	401 Unauthorized	Pass
Create article without a token	401 Unauthorized	Pass
Author creates an article	201, status draft	Pass
Draft appears in public listing	No	Pass
Author edits own draft	200 OK	Pass
Author edits another author's draft	403 Forbidden	Pass
Author tries to publish	403 Forbidden	Pass
Editor publishes a draft	200 OK, status published	Pass
Edit a published article	403 Forbidden	Pass
Published article appears in public listing	Yes	Pass
Filter by category and search by keyword	Only matching published articles	Pass

All four expected outcomes of the problem statement were achieved: authors create and edit only their own drafts, only editors publish, readers can search and filter by category, and drafts never appear in the public listing.

6.2 Limitations

- Users choose their own role when they register, so anyone can sign up as an editor. A production system would use an admin-controlled role assignment or an invitation flow.
- The JWT is kept in browser local storage, which is exposed to cross-site scripting attacks. HTTP-only cookies would be more secure.
- Search uses pattern matching, which is slow on large collections. A MongoDB text index would perform better.
- The article list has no pagination, and there is no rate limiting on the login endpoint.

6.3 Future scope

Admin-controlled role assignment, an editor "reject with feedback" step, pagination, MongoDB text indexes, article images, refresh tokens, and automated tests for the API.

6.4 Conclusion

The project shows how a status-driven workflow and layered authorization can protect content in a multi-user application. Keeping authentication, role checks and ownership checks as separate, reusable middleware kept the controllers simple and made the security rules easy to test. The result is a working, deployed platform that meets all the objectives of the case study.

7. References

1. Node.js Documentation. <https://nodejs.org/en/docs>
2. Express.js Documentation. <https://expressjs.com>
3. MongoDB Atlas Documentation. <https://www.mongodb.com/docs/atlas/>
4. Mongoose Documentation. <https://mongoosejs.com/docs/>
5. React Documentation. <https://react.dev>
6. Vite Documentation. <https://vite.dev>
7. React Router Documentation. <https://reactrouter.com>
8. Axios Documentation. <https://axios-http.com/docs/intro>
9. Jones, M., Bradley, J., and Sakimura, N. "JSON Web Token (JWT)." RFC 7519, IETF, 2015.
10. Provos, N. and Mazières, D. "A Future-Adaptable Password Scheme." Proceedings of the USENIX Annual Technical Conference, 1999.
11. Render Documentation. <https://render.com/docs>
12. Vercel Documentation. <https://vercel.com/docs>
13. ITM Skills University, B.Tech CSE. "Backend Development Case Study: Node.js / Express / MongoDB," Case Study 141.